

PHƯƠNG PHÁP THỬ SAI (Trial and Error)

PGS.TS. Đặng Thị Oanh

Ngày 9 tháng 3 năm 2026

Tài liệu tham khảo

Chapter 3, section 12.1, 12.2

Anany Levitin, **Introduction to the Design and Analysis of Algorithms**, ebook.

Review the Previous Lesson

- ❶ Chiến lược chia để trị
- ❷ Mô hình, lược đồ chia để trị
- ❸ Ưu, nhược điểm của chia để trị
- ❹ Một số Bài toán giải bằng chia để trị

Nội dung

- 1 Giới thiệu về lớp phương pháp thử sai
- 2 Phương pháp vét cạn
- 3 Phương pháp quay lui
- 4 Phương pháp nhánh cận
- 5 Bài tập

Lớp phương pháp thử sai

Duyệt tất cả các phương án tồn tại nghiệm của bài toán để xác định nghiệm đúng hoặc đủ tốt.

- ➊ Ưu điểm: Tìm thấy lời giải.
- ➋ Nhược điểm: Tốn thời gian.

Phương pháp vét cạn

Là một phương pháp giải quyết vấn đề bằng cách kiểm tra tất cả các khả năng hoặc giải pháp có thể có một cách có hệ thống cho đến khi tìm thấy một giải pháp đúng hoặc đã kiểm tra hết tất cả các trường hợp.

Brute force template:

$D = (x_1, x_2, \dots, x_n)$: Solution domain (tập khả năng)

$x_i \in D_i$, where $D_i = \{d_{ij} | j = 1 \dots m_i\}$ – finite sets

Quy tắc xác định lời giải

$f : D \rightarrow \{true, false\}$

Solution: $x = (x_1, x_2, \dots, x_n)$ such $f(x) = true$

Phương pháp vét cạn

Brute force diagram

BruteForce(D) $\equiv$

```
for ( $x_1 = d_{11} \dots d_{1m1}$ )  
    for ( $x_2 = d_{21} \dots d_{2m2}$ )  
        ...  
            for ( $x_n = d_{n1} \dots d_{nmn}$ )  
                if ( $f(x_1, x_2, \dots x_n)$ )  
                    <Solution  $x = (x_1, x_2, \dots x_n)$  >;  
end.
```

Phương pháp vét cạn

Ví dụ

- **Bài toán gà, chó:**

Vừa gà vừa chó; Bó lại cho tròn;

Ba mươi sáu con; Một trăm chân chẵn

Hỏi số gà, số chó?

- **Phân tích:**

- Gọi x, y là số gà, chó:

$$x \in D_1 = \{1, 2, \dots, 36\}, y \in D_2 = \{1, 2, \dots, 36\}$$

- Không gian nghiệm của bài toán: $D = \{(x, y)\}$

- Quy tắc xác định lời giải:

$$f(x, y) = (x + y = 36) \text{ và } (2x + 4y = 100)$$

- Nghiệm của bài toán:

$$x = (x, y) \text{ sao cho } f(x, y) = \text{true}$$

Phương pháp vét cạn

Lược đồ

ChickenDog $\equiv$

for ($x = 0 \dots 36$)

for ($y = 0 \dots 36$)

if ($((x + y) = 36 \ \& \ (2x + 4y) = 100)$
 $< output(x, y) >;$

end.

- **Time complexity:** $O(n^2)$, $n = 36$ – depend on problem space
- **Improve:**
 - *Reduce problem space:*
 $D = \{(x)\}, x \in D_1 = \{1, 2, \dots, 25\}$
 - *Solution rule:* $f(x) = (4x + 2(36 - x) = 100)$
 - *Time complexity:* $O(n)$, $n = 25$

Phương pháp vét cạn

Ưu điểm

- **Đảm bảo tìm ra giải pháp (nếu tồn tại):** Đây là ưu điểm lớn nhất. Miễn là vấn đề có giải pháp và có đủ thời gian/tài nguyên để duyệt hết, thuật toán vét cạn chắc chắn sẽ tìm thấy nó.
- **Dễ hiểu và dễ triển khai:** Logic đơn giản, thường không yêu cầu kiến thức phức tạp về thuật toán hay cấu trúc dữ liệu đặc biệt.
- **Điểm khởi đầu:** Thường được sử dụng làm phương pháp cơ sở để kiểm tra tính đúng đắn của các thuật toán phức tạp hơn hoặc làm cơ sở để so sánh hiệu suất.
- **Tính tổng quát:** Áp dụng được cho nhiều loại vấn đề khác nhau mà không cần nhiều thay đổi.

Phương pháp vét cạn

Nhược điểm

- **Hiệu suất kém (độ phức tạp thời gian cao):** Đây là nhược điểm chí mạng của thuật toán vét cạn. Với không gian tìm kiếm lớn, thời gian chạy có thể tăng theo cấp số mũ hoặc giai thừa, khiến nó không khả thi trong thực tế.
- **Tiêu tốn tài nguyên:** Cần nhiều bộ nhớ và CPU để lưu trữ và kiểm tra tất cả các khả năng.
- **Không tối ưu:** Mặc dù tìm ra giải pháp, nhưng có thể không phải là giải pháp tối ưu nhất nếu vấn đề yêu cầu tối ưu hóa (ví dụ: đường đi ngắn nhất).

Phương pháp vét cạn

Ứng dụng thực tế

- **Tạo khóa hoặc mật khẩu ngẫu nhiên:** Trong các hệ thống cũ hoặc cho các mục đích không quá nhạy cảm, có thể sử dụng vét cạn để tìm kiếm khóa ngẫu nhiên. (Tuy nhiên, đối với bảo mật hiện đại, điều này rất nguy hiểm).
- **Giải các bài toán đồ nhỏ:**
 - Sudoku: Thử tất cả các số có thể vào ô trống.
 - N-Queens Problem (với N nhỏ): Đặt N quân hậu trên bàn cờ $N \times N$ sao cho không có hai quân hậu nào ăn nhau.
 - Mê cung: Tìm tất cả các đường đi có thể để thoát khỏi mê cung.

Phương pháp vét cạn

Ứng dụng thực tế (tiếp)

- **Kiểm tra tính đúng đắn của các thuật toán khác:** Khi phát triển thuật toán tối ưu hơn, có thể sử dụng thuật toán vét cạn trên các bộ dữ liệu nhỏ để xác minh rằng thuật toán tối ưu của ta đưa ra cùng kết quả đúng.
- **Tìm kiếm trong các tập hợp nhỏ:** tìm giá trị lớn nhất/nhỏ nhất trong một danh sách nhỏ, hoặc kiểm tra xem một phần tử có tồn tại trong một tập hợp không sắp xếp.
- **Tạo ra tất cả các hoán vị/tổ hợp:** Phát sinh tất cả các hoán vị (permutations) hoặc tổ hợp (combinations) của một tập hợp các phần tử.
- **Cracking mã hóa (với khóa yếu):** Tấn công brute-force vào các hệ thống mã hóa sử dụng khóa ngắn hoặc thuật toán yếu. (Đây là mục đích tấn công, không phải ứng dụng xây dựng).

Phương pháp vét cạn

Khi nào nên sử dụng vét cạn?

- **Kích thước dữ liệu nhỏ:** Nếu không gian tìm kiếm nhỏ và chắc chắn rằng thuật toán sẽ chạy trong thời gian chấp nhận được.
- **Yêu cầu tính đúng đắn tuyệt đối:** Khi cần đảm bảo tìm thấy tất cả các giải pháp hoặc một giải pháp chắc chắn đúng và không có rủi ro bỏ sót.
- **Kiểm thử và đánh giá:** Để kiểm tra tính đúng đắn của các thuật toán phức tạp hơn hoặc để thiết lập một đường cơ sở hiệu suất.
- **Không có thuật toán hiệu quả hơn được biết đến:** Đối với một số vấn đề mới hoặc chưa được nghiên cứu kỹ, vét cạn có thể là cách duy nhất để bắt đầu.

Phương pháp vét cạn

Kết luận

- Thuật toán vét cạn là một phương pháp giải quyết vấn đề bằng cách **kiểm tra một cách toàn diện mọi khả năng có thể**.
- Mặc dù hiệu suất kém, nhưng **sự đơn giản và khả năng đảm bảo tìm ra giải pháp (nếu tồn tại)** → *công cụ có giá trị trong một số trường hợp cụ thể*.
- Thuật toán vét cạn là nền tảng để từ đó nhiều kỹ thuật tối ưu hóa đã được phát triển.
- Hiểu rõ thuật toán vét cạn, ngoài việc giúp giải quyết bài toán, còn là bước đầu tiên để đánh giá độ phức tạp của bài toán và tìm kiếm những giải pháp thông minh hơn.

Phương pháp vét cạn

Other problems:

- Selection sort and Bubble sort – Brute force
- Searching – Brute force :
 - Sequential search (tìm kiếm phần tử trong dãy)
 - String matching (khớp nhau)
 - Closest pair of point (cặp điểm gần nhất)
 - Convex hull (bao lồi)

Anany's book, section 3.1, 3.2, 3.3

Phương pháp quay lui (backtracking)

Ý tưởng

- 1 Quay lui là một kỹ thuật thiết kế giải thuật **dựa trên đệ quy**. Ý tưởng của quay lui là tìm lời giải từng bước, mỗi bước chọn một trong số các lựa chọn khả dĩ (*theo nguyên tắc vét cạn - duyệt vét cạn*) và đệ quy.

Tại mỗi bước, nếu có một lựa chọn được chấp thuận thì ghi nhận lại lựa chọn này và tiến hành các bước thử tiếp theo. Còn ngược lại không có lựa chọn nào thích hợp thì **quay lại bước trước** → **quay lui**.

- 2 Thuật toán quay lui dùng để giải bài toán liệt kê các cấu hình. Mỗi cấu hình được xây dựng bằng cách xây dựng từng phần tử, mỗi phần tử được chọn bằng cách thử tất cả các khả năng.
- 3 Thuật ngữ "backtrack" do nhà toán học người Mỹ D. H. Lehmer đề ra (1950).

Phương pháp quay lui (backtracking)

Mô hình

Không gian nghiệm của bài toán (tập khả năng) $D = \{(x_1, x_2, \dots, x_n)\}$, gồm các cấu hình liệt kê có dạng $(x_1, x_2, \dots, x_n)$ cần được xây dựng:

- Xét tất cả các giá trị x_1 có thể nhận, thử x_1 nhận các giá trị đó. Với mỗi giá trị của x_1 ta sẽ:
- Xét tất cả giá trị x_2 có thể nhận, lại thử x_2 cho các giá trị đó. Với mỗi giá trị của x_2 lại xét khả năng giá trị của $x_3 \dots$ tiếp tục như vậy cho tới bước:
- ...
- Xét tất cả giá trị x_n có thể nhận, thử cho x_n nhận lần lượt giá trị đó.
- $\Rightarrow$ cấu hình tìm được $(x_1, x_2, \dots, x_n)$.
- *Tại mỗi bước thứ i :*
 - Xây dựng phần tử x_i
 - Xác định x_i theo khả năng v .
 - Nếu $i = n$, ta có được một lời giải, ngược lại thì tiến hành Bước $i + 1$ để xác định x_{i+1} . Nếu không có bất kỳ khả năng nào chấp nhận được cho x_i , thì lại lùi lại bước trước để xác định lại phần tử x_{i-1} .

Bản chất của quay lui là một quá trình tìm kiếm theo chiều sâu (Depth-First Search).

Phương pháp quay lui (backtracking)

Lược đồ phương pháp

Try (i) $\equiv$ //sinh phần tử Thứ i của cấu hình

for ($v \in$ tập khả năng phần tử nghiệm x_i)

if (x_i chấp nhận được giá trị v)

$x_i = v$;

<Ghi nhận trạng thái chấp nhận v >;

if ($i = n$) //đủ n phần tử của cấu hình đã xác định

<ghi nhận nghiệm>;

else //lời gọi sinh phần tử tiếp theo của cấu hình

Try ($i + 1$)

<Khôi phục trạng thái chưa chấp nhận v >;

endif;

endfor

End.

Phương pháp quay lui (backtracking)

Bài toán 1: Liệt kê dãy nhị phân độ dài n

- Phân tích:

- Dãy nhị phân $(x_1x_2 \dots x_n)$ trong đó $x_i = 0, 1$;
- Dùng giải thuật **Try**(i) để sinh giá trị x_i ;
- Nếu $i = n$, in giá trị nghiệm; ngược lại, sinh tiếp x_{i+1} bằng **Try**($i + 1$).

Phương pháp quay lui (backtracking)

Bài toán 1: Liệt kê dãy nhị phân độ dài n

• Phân tích:

- Dãy nhị phân $(x_1 x_2 \dots x_n)$ trong đó $x_i = 0, 1$;
- Dùng giải thuật **Try**(i) để sinh giá trị x_i ;
- Nếu $i = n$, in giá trị nghiệm; ngược lại, sinh tiếp x_{i+1} bằng **Try**($i + 1$).

• Lược đồ:

Try(i) $\equiv$

for ($v = 0 \dots 1$) // v nhận giá trị 0 hoặc 1

$x_i = v$;

if ($i = n$) *printResult* (x);

else **Try**($i + 1$);

endfor;

End.

Lời gọi ban đầu **Try**(1).

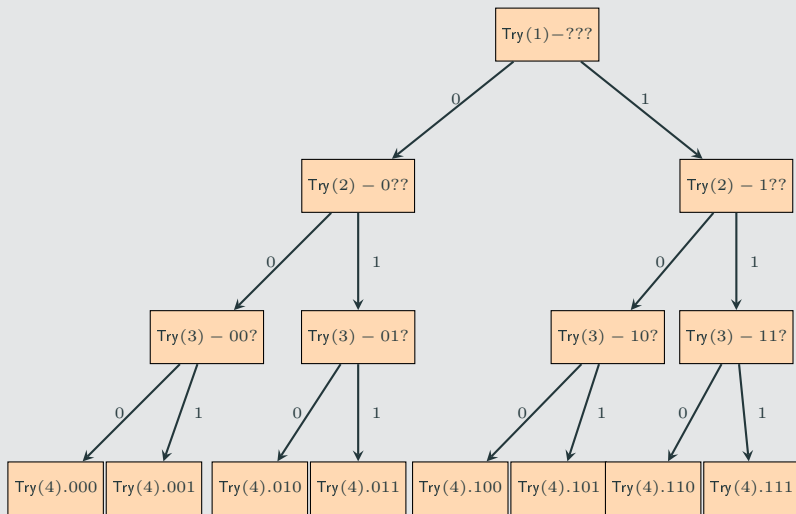
Phương pháp quay lui (backtracking)

VD: Tìm danh sách các dãy nhị phân độ dài 3

- **Xét mọi trường hợp ký tự thứ nhất.** Ta được 0 hoặc 1.
- **Với mỗi trường hợp của ký tự thứ nhất,** xét tiếp mọi trường hợp ký tự thứ hai. Ta được 00, 01 *từ 0 ở bước trước* và 10, 11 *từ 1 ở bước trước*.
- Tương tự, **với mỗi trường hợp của ký tự thứ hai,** ta xét nốt mọi trường hợp ở ký tự thứ ba. Các dãy nhận được là 000, 001, 010, 011, 100, 101, 110, 111.

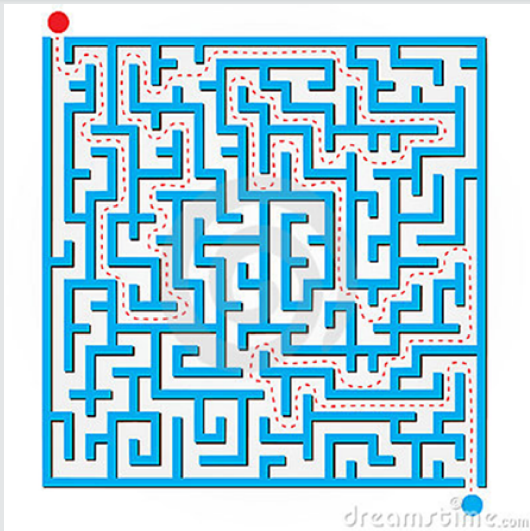
Phương pháp quay lui (backtracking)

VD: Tìm danh sách các dãy nhị phân độ dài 3
Hoạt động của thuật toán



Phương pháp quay lui (backtracking)

Bài toán 2: Tìm đường đi trong mê cung



Phương pháp quay lui (backtracking)

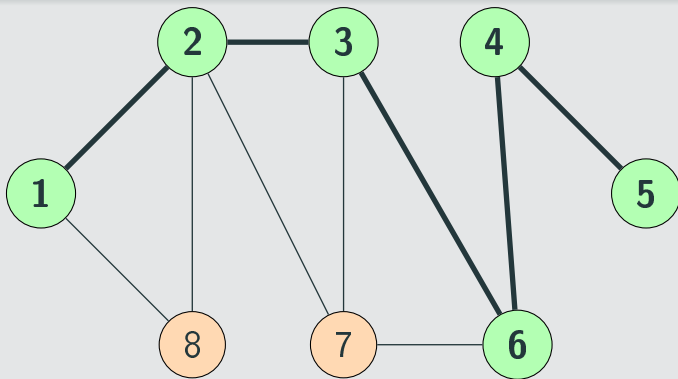
Bài toán 2: Tìm đường đi trong mê cung

Phân tích: Xét mê cung như một đồ thị vô hướng $G = (V, E)$

- Mỗi phòng của mê cung là một nút của đồ thị;
- Mỗi đường đi giữa các phòng là một cạnh nối giữa các nút của đồ thị;
- Đỉnh xuất phát: $S \in V$, đỉnh kết thúc $F \in V$;
- Đường đi tìm được là dãy $S = x_1, x_2, \dots, x_k = F$ với $x_i \in V, (x_i, x_{i+1}) \in E$

Phương pháp quay lui (backtracking)

Bài toán 2: Tìm đường đi trong mê cung



Ví dụ: $S = 1, F = 5 \in V$

$\Rightarrow$ Đường đi: $S = 1 \rightarrow 2 \rightarrow 3 \rightarrow 6 \rightarrow 4 \rightarrow 5 = F$

Phương pháp quay lui (backtracking)

Bài toán 2: Tìm đường đi trong mê cung

Ý tưởng thuật toán (dùng quay lui)

- Từ đỉnh xuất phát, hình thành dần các đỉnh trên đường đi tới đỉnh đích.
- Dùng giải thuật **Try**(i) để sinh giá trị x_i
 - Đỉnh v được chấp nhận cho x_i nếu v chưa được đi qua, có cạnh (x_{i-1}, v)
 - Nếu $x_i = F$ thì kết thúc, in đường đi tìm được
 - Ngược lại sinh tiếp x_{i+1} bằng **Try**($i + 1$)
- **Lời gọi ban đầu**
 - $x_1 = S$
 - **Try**(2)

Phương pháp quay lui (backtracking)

Bài toán 2: Tìm đường đi trong mê cung

Mô tả dữ liệu

- **Input:** Đồ thị G , đỉnh S , F
 - n đỉnh của đồ thị được đánh số từ $1, \dots, n$
 - Ma trận kề $M = (m_{ij})$ cỡ $n \times n$ của đồ thị
 - $m_{ij} = 1$ nếu có cạnh nối đỉnh i với đỉnh j ,
 - $m_{ij} = 0$ nếu ngược lại.
- **Output:** đường đi $S = x_1, x_2, \dots, x_k = F$
- **Sử dụng biến:**

Mảng: *Boolean* $\text{Daqua}[i : 1 \dots n]$ đánh dấu đỉnh i đã được đi qua hay chưa?

 - Khởi tạo $\text{Daqua}[i : 1 \dots n] = \text{false}$.

Phương pháp nhánh cận

Bài toán 2: Tìm đường đi trong mê cung

Try(i) $\equiv$

for ($v = 1 \dots n$) *// duyệt qua các đỉnh*

if ($m[x_{i-1}, v] = 1$) & (not *Daqua*[v]) *// v chấp nhận được*

$x_i = v$;

Daqua[v] = true; *// ghi nhận trạng thái đã chọn v*

if ($x_i = F$) *printResult* ($x_1, x_2, \dots, x_i$);

else **Try**($i + 1$);

Daqua[v] = false; *// khôi phục trạng thái chưa chọn v*

endif;

endfor;

End.

Lời gọi ban đầu: $x_1 = S$; **Try**(2).

Phương pháp quay lui (backtracking)

Bài toán 3: Phân công công việc

- Có n người với n công việc, người thứ i làm việc thứ j với năng suất $s[i, j]$. Tìm phương án phân công việc cho mỗi người để tổng năng suất lớn nhất.

- Ý tưởng thuật toán (dùng quay lui):

- Một phương án phân chia công việc $\equiv$ 1 cấu hình liệt kê (người-việc)
- Dùng giải thuật Try(i) để sinh giá trị w_i là công việc phân cho người i
 - Công việc v được chấp nhận cho w_i nếu v chưa được phân công, và người i có thể làm việc v .
 - Nếu $i = n$ thì kết thúc, xử lí phương án tìm được:
 - Tính tổng hiệu suất của phương án.
 - So sánh với hiệu suất max đã tính được. Cập nhật nếu cần.
 - Ngược lại ($i < n$) sinh tiếp w_{i+1} bằng Try($i + 1$).

Phương pháp quay lui (backtracking)

Bài toán 3: Phân công công việc

Mô tả dữ liệu

- **Input**

- n người được đánh chỉ số $1, \dots, n$, n công việc được đánh chỉ số $1, \dots, n$
- Mảng double $s[i : 1 \dots n, j : 1 \dots n]$ hiệu suất của người i làm việc j

- **Output**

- *Double* $S_{\max}$ hiệu suất công việc cao nhất
- *int* $w_{\max}[i : 1 \dots n]$ công việc phân cho người thứ i (phương án đạt $S_{\max}$)

- **Sử dụng biến:**

Mảng: *boolean* $used[j : 1 \dots n]$ đánh dấu việc j đã phân công (true) hay chưa. *int* $w[i : 1..n]$ công việc phân cho người thứ i (*phương án hình thành*).

Phương pháp quay lui (backtracking)

Bài toán 3: Phân công công việc

Try(i) $\equiv$

for ($v = 1 \dots n$) *//duyet qua các công việc*

if (not $used[v]$ & $s[i, v] < -1$) *//v chấp nhận được*

$w[i] = v$;

$used[v] = \text{true}$; *//ghi nhận trạng thái đã chọn v*

if ($i = n$)

$\text{Xử lí phương án } w[1..n]$; *//Tính tổng hiệu suất,
cập nhật Smax, wmax nếu lớn hơn*

else Try(i+1) ;

$used[v] = \text{false}$; *//khôi phục trạng thái chưa chọn v*

endif;

endfor;

End.

Lời gọi ban đầu: $S_{\max} = 0$; Try(1);

Phương pháp quay lui (backtracking)

Backtracking applications

- Hamiltonian Path Problem (Chu trình Hamilton của đồ thị)
- Traverling Salesman Problem (Người đưa hàng)
- Knapsack Problem (Xếp balo)
- Map Coloring Problem (Tô màu bản đồ)
- Problem of placing chess (queen, rooks, knights) on chessboard (Bài toán xếp quân cờ: hậu, xe, mã).
- ...

Phương pháp nhánh cận

Optimal problem (Bài toán tối ưu)

- Bài toán yêu cầu tìm ra phương án tốt nhất thỏa mãn một số yêu cầu ràng buộc nào đó – nghiệm của bài toán đạt giá trị max/min trong không gian nghiệm.
 - Thuộc lĩnh vực Tối ưu toán học/Quy hoạch toán học. **Lời giải toán có thể khó $\Rightarrow$ Sự vào cuộc của Tin học.**
 - **Hai hướng tiếp cận tìm lời giải tối ưu cho bài toán:**
 - Tìm từng lời giải, khi xác định được một lời giải thì so sánh chi phí của nó với chi phí tốt nhất hiện có. Nếu tốt hơn thì cập nhật chi phí tốt nhất.
 - Với mỗi lời giải, khi xây dựng các thành phần nghiệm luôn kiểm tra điều kiện: nếu đi tiếp theo hướng này thì có khả năng nhận được lời giải tốt hơn lời giải hiện có không? Nếu không thì thôi không đi theo hướng này nữa.
- $\Rightarrow$ Nguyên lí nhánh cận (Branch and Bound).**

Phương pháp nhánh cận

Nguyên lý nhánh cận

- **Nhánh cận** : Thuật toán tìm lời giải cho các bài toán tối ưu dạng liệt kê cấu hình dựa trên nguyên lý đánh giá nhánh cận.
- **Nguyên lý đánh giá nhánh cận**: Sử dụng các thông tin đã tìm được trong lời giải của bài toán để loại bỏ sớm phương án không dẫn tới lời giải tối ưu.
- **Bản chất**: Sử dụng phương pháp quay lui nhưng tại mỗi bước đưa thêm thao tác **đánh giá** giá trị phương án hiện có, **cụ thể là**:
 - **Nếu là phương án tối ưu hoặc có hy vọng trở thành phương án tối ưu** (tức là tốt hơn phương án hiện có), thì cập nhật lại phương án tối ưu hoặc đi tiếp theo hướng đó.
 - **Ngược lại thì bỏ qua hướng đang xét.**

Phương pháp nhánh cận

Mô hình

- Không gian của bài toán (tập khả năng) $D = \{(x_1, x_2, \dots, x_n)\}$ gồm các cấu hình liệt kê có dạng $(x_1, x_2, \dots, x_n)$. Mỗi cấu hình x , xác định một giá trị hàm chi phí $f(x)$:

$$f : D \rightarrow Z \text{ sao cho } f(x) = C (C \in Z)$$

Nghiệm của bài toán: $x = (x_1, x_2, \dots, x_n)$ sao cho $f(x) = \text{giá trị tối ưu (max/min)}$.

- Cho x_i nhận lần lượt các giá trị có thể. Với mỗi giá trị thử gán cho x_i xét khả năng chọn $x_{i+1}, x_{i+2} \dots$
- **Tại mỗi bước i :** Xây dựng thành phần x_i
 - Xác định x_i theo khả năng v .
 - Tính chi phí lời giải nhận được. Nếu “tốt hơn” lời giải hiện thời, thì chấp nhận x_i theo khả năng v . Tiếp tục xác định $x_{i+1}, \dots$ đến khi gặp nghiệm.
 - Nếu không có bất kỳ khả năng nào chấp nhận được cho x_i hoặc lời giải xấu hơn thì lùi lại bước trước để xác định lại thành phần x_{i-1} .

Phương pháp nhánh cận

Thuật toán

Try(i, S) $\equiv$ //Sinh thành phần thứ i của cấu hình với chi phí hiện thời S
for (v thuộc tập khả năng thành phần nghiệm x_i)

if (v là chấp nhận được)

$T = S +$ Chi phí nghiệm khi có thêm thành phần v ;

if (T tốt hơn $T_{optimize}$) //Tốt hơn chi phí tốt nhất hiện có

$x_i = v$;

 <Ghi nhận trạng thái chấp nhận v >;

if (x_i là trạng thái kết thúc)

 <Ghi nhận nghiệm>;

$T_{optimize} = T$; //Cập nhật chi phí tốt nhất

else **Try**($i+1, T$); //sinh T .phần tiếp theo với chi phí hiện thời T

endif;

 <Khôi phục trạng thái chưa chấp nhận v >;

endif;

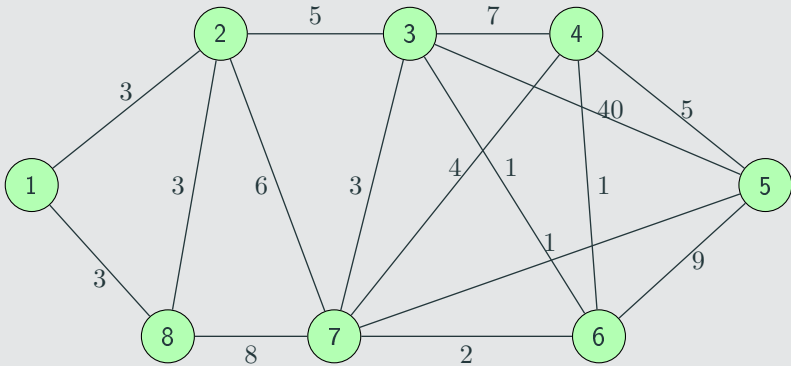
endif;

End.

Phương pháp nhánh cận

Traveling Salesman – Bài toán người đưa hàng

Người bán hàng trên hệ thống n thành phố. Giữa các thành phố có thể **có/không** đường đi có chi phí cho trước. Hãy xác định một hành trình để đi từ một thành phố tới tất cả các thành phố còn lại 1 lần và quay về sao cho tổng **chi phí là nhỏ nhất**.



Phương pháp nhánh cận

Mô tả – Bài toán người đưa hàng

Phân tích: Biểu diễn mạng lưới giao thông giữa các thành phố như một đồ thị có trọng số $G = (V, E)$

- Mỗi thành phố là một nút của đồ thị (đánh số $1, \dots, n$)
- Mỗi đường đi giữa các thành phố là một cạnh nối giữa các nút của đồ thị, có thể **có/vô** hướng, trên đó có ghi trọng số là chi phí đường đi. Các cặp cạnh không có đường đi trọng số là ∞
- Đỉnh xuất phát $\equiv$ kết thúc: $S \in V$
- Đường đi tìm được là dãy $S = x_1, x_2, \dots, x_n, x_1 = S$ với $x_i \in V, (x_i, x_{i+1}) \in E$, có tổng chi phí nhỏ nhất

$\Rightarrow$ Sinh các dãy hoán vị $1 \dots n$ và tính dãy có chi phí nhỏ nhất:

- Quay lui
- Nhánh cận

Phương pháp nhánh cận

Mô tả – Bài toán người đưa hàng

Phân tích: Tiếp cận theo phương pháp nhánh cận

- Chi phí tốt nhất tìm được (**BestCost**). Đầu tiên **BestCost** = $+\infty$.
- Tại mỗi bước chọn x_i : Chi phí đường đi từ x_1 đến x_{i-1} là C .
 - Với mỗi khả năng v , tính chi phí $C_1 = C +$ chi phí từ x_{i-1} tới v
 - Nếu C_1 xấu hơn (lớn hơn) **BestCost** hoặc không có khả năng nào chấp nhận được cho x_i , thì lùi lại bước trước để xác định lại thành phần x_{i-1} .
 - Nếu C_1 tốt hơn (nhỏ hơn) **BestCost** thì chấp nhận x_i theo khả năng v . Tiếp tục xác định $x_{i+1}, \dots$
- Đến khi gặp nghiệm ($i = n + 1$ & $x_i = S$) :
 - Cập nhật đường đi tốt nhất hiện tại đã tìm được
 - Cập nhật giá trị **BestCost** mới: **BestCost** = C_1
- Kết thúc tìm kiếm nếu **BestCost** = $+\infty$
 $\Rightarrow$ không có đường đi.

Phương pháp nhánh cận

Mô tả – Bài toán người đưa hàng

Mô tả dữ liệu

- Đánh chỉ số các đỉnh của đồ thị từ $1 \dots n$
- Biểu diễn đồ thị G bằng ma trận kề $M = (c_{ij})$ cỡ $n \times n$
- $c_{ij} = cost$ nếu có cạnh nối đỉnh i với đỉnh j với chi phí $cost$
 $= \infty$ nếu không có đường đi.
- **Mảng:** $Daqua[1 \dots n]$ đánh dấu đỉnh i đã được đi qua trên đường đi hay chưa?
 $Daqua[i] = true$ nếu đỉnh i đã có trên đường đi
 $= false$ nếu ngược lại, đỉnh i chưa có trên đường đi
Khởi tạo: $Daqua[1 \dots n] = false$.

Phương pháp nhánh cận

Diagram – Bài toán người đưa hàng

$\text{Try}(i, C) \equiv //$ Sinh thành phần thứ i của cấu hình với chi phí hiện thời C

for ($v = 1 \dots n$)

if ($c[x_{i-1}, v] < \infty$) & (not $\text{Daqua}[v]$))

$C_1 = C + c[x_{i-1}, v];$

if ($C_1 < \text{BestCost}$) *//Tốt hơn chi phí tốt nhất hiện có*

$x_i = v;$

$\text{Daqua}[v] = \text{true};$

if ($i = n + 1$) & ($x_i = S$)

$< \text{Ghi nhận nghiệm } x_1, x_2 \dots x_{n+1} >;$

$\text{BestCost} = C_1; //$ Cập nhật chi phí tốt nhất

else if ($i \leq n$)

$\text{Try}(i + 1, C_1); //$ sinh thành phần tiếp theo với chi phí hiện thời C_1

endif;

$\text{Daqua}[v] = \text{false};$

endif;

endif;

End.

Phương pháp nhánh cận

Branch and Bound applications

- Knapsack problem (Xếp ba lô)
- Integer programming (Quy hoạch nguyên)
- Nonlinear programming (Quy hoạch phi tuyến)
- Maximum satisfiability problem (MAX-SAT - Bài toán thỏa được cực đại)
- Nearest neighbor search (NNS - Láng giềng gần nhất)
- Cutting stock problem

Bài tập

- Program the designed algorithms
- Other examples or application